

Entwicklerdokumentation für SaS3

Stand der Dokumentation

- Letzte inhaltliche Aktualisierung: 2026-04-14
- Quelle: aktueller Code-Stand im Repository
- Ergänzendes Fachbegriff-Glossar aus der Projektdokumentation: `doku/Glossar.md`

1. Ziel und Umfang

Diese Dokumentation beschreibt die technische Architektur und die wichtigsten Funktionen der Anwendung SaS3 auf Basis des aktuellen Codes.

Zielgruppe:

- Backend-Entwickler
- Frontend-Entwickler
- Betrieb/Administration

Abgedeckte Kernbereiche:

- Login, Session und Autorisierung
- Schadensfall-Workflow
- Datenbanklayer und Schema
- Kataloge (Material, Versicherung, Kasse)
- Admin-Funktionen (Rechte, DB-Konfiguration)
- Frontend-Seiten und deren API-Anbindung

2. Systemüberblick

Zentrale Komponenten:

- `server.mjs`: Express-Server, Routing, Login, Workflow, Guards
- `src/db.js`: PostgreSQL-Zugriffe, Schema-Setup, Mapping-Funktionen
- `src/rights.js`: Rollenmodell, Rechteableitung, Sicht-/Editierprüfungen
- `src/LDAPSearch.js`: LDAP-Suchen für AD-Authentifizierung und Gruppen
- `src/create-damage-case.js`: Client-Logik für Schadensfallmaske
- `public/*.html`: UI-Seiten
- `config/config.json`: Server-, LDAP-, DB-, Rollen- und Testuser-Konfiguration

Laufzeitmodus:

- `Scope main`: Produktiv-DB
- `Scope dummy`: Dummy/Test-DB

Der Scope wird pro Request gesetzt:

```
app.use((req, _res, next) => {
  const scope = req.session?.user?.isDummy ? "dummy" : "main";
  db.runWithScope(scope, next);
});
```

3. Start und Betrieb

3.1 Startkommando

```
{
  "scripts": {
    "start": "node server.mjs"
  }
}
```

3.2 HTTP/HTTPS-Start mit Zertifikatserkennung

```
function start() {
  const certPath = "/etc/lbm/ssl/privkey.pem";
  const fullchainPath = "/etc/lbm/ssl/fullchain.pem";

  const hasCerts = fs.existsSync(certPath);

  if (hasCerts) {
    https.createServer(sslOptions, app).listen(config.server.port, () => {
      connectDatabaseWithRetry();
    });
  } else {
    http.createServer(app).listen(config.server.port, () => {
      connectDatabaseWithRetry();
    });
  }
}
```

3.3 DB-Connect mit Retry

```
async function connectDatabaseWithRetry() {
  const retryDelayMs = 15000;

  try {
    await db.connect();
    console.log("Datenbankverbindung hergestellt.");
  } catch (error) {
    setTimeout(connectDatabaseWithRetry, retryDelayMs);
  }
}
```

4. Authentifizierung und Session

4.1 Login-Typen

- Dummy-Login über `config.testUsers`
- AD-Login über LDAP mit Benutzername + Passwort

4.2 Benutzername-Normalisierung

```
function normalizeLoginUsername(value) {  
  const raw = String(value || "").trim();  
  const withoutDomainPrefix = raw.includes("\\") ? raw.split("\\").pop() : raw;  
  const withoutUpnSuffix = withoutDomainPrefix.includes("@")  
    ? withoutDomainPrefix.split("@")[0]  
    : withoutDomainPrefix;  
  
  return withoutUpnSuffix.trim();  
}
```

Akzeptierte Eingaben:

- domain\\user
- user@domain
- user

4.3 AD-Credential-Prüfung

```
const credentialCheck = new LDAPSearch(  
  config.ldap.realm,  
  normalizedUsername,  
  normalizedPassword,  
);  
  
const filter = config.ldap.userFilter.replace("{{username}}", normalizedUsername);  
const authResults = await credentialCheck.search(filter, "dn");
```

4.4 Session-Regeneration (Fixation-Schutz)

```
await new Promise((resolve, reject) => {  
  req.session.regenerate((error) => {  
    if (error) return reject(error);  
    req.session.user = result.user;  
    resolve();  
  });  
});
```

4.5 Login-Endpunkte

- GET /api/auth-options: Dummy-Accounts für Loginseite
- POST /login: Login
- GET /api/me: Session-User
- GET /logout: Session beenden

5. Rollen- und Rechtesystem

Rechte werden aus LDAP-Gruppen und `config.rights` berechnet.

5.1 Rollenvererbung

```
function expandRole(roleName, roleDefinitions, visited = new Set()) {
  if (!roleName || visited.has(roleName)) return [];

  visited.add(roleName);
  const definition = roleDefinitions[roleName] || {};
  const inheritedRoles = asArray(definition.inherits);
  const permissions = new Set(asArray(definition.permissions));

  inheritedRoles.forEach((inheritedRole) => {
    expandRole(inheritedRole, roleDefinitions, visited).forEach((permission) => {
      permissions.add(permission);
    });
  });

  return Array.from(permissions);
}
```

5.2 Kernfunktionen in `src/rights.js`

- `resolveAuthorization(memberDns, config)`
- `hasPermission(user, permission)`
- `hasAnyPermission(user, permissions)`
- `canViewCase(user, damageCase)`
- `canEditCase(user, damageCase)`
- `isOwner(user, damageCase)`

5.3 Statusabhängige Sichtbarkeit/Bearbeitung

```
if (hasPermission(user, "approve_case") && status === "Leitung") {
  return true;
}
```

Das Rechtekonzept ist nicht nur rollenbasiert, sondern auch workflow-/statusbasiert.

6. Datenbankschicht (`src/db.js`)

6.1 Kernprinzipien

- Pool-Caching pro Scope (`main, dummy`)
- automatisches Schema-Setup beim ersten Zugriff
- Mapping DB -> API-Objekt
- Soft-Delete bei Katalogen/Positionen

6.2 Scope via `AsyncLocalStorage`

```
const dbScopeStorage = new AsyncLocalStorage();

function runWithScope(scope, fn) {
  const normalizedScope = scope === "dummy" ? "dummy" : "main";
  return dbScopeStorage.run({ scope: normalizedScope }, fn);
}
```

6.3 Wichtige Tabellen

- SYSBENUTZER
- DATSCHADENSFAELLE
- SYSKATALOG
- SFOKATALOGPOSITIONEN
- SYSVERSICHERUNGEN
- SYSKASSEN

6.4 Schlüsselfunktionen

Benutzer:

- upsertUser(user)
- getUsers()

Schadensfälle:

- createDamageCase(damageCase)
- listDamageCases(options)
- getDamageCaseById(id)
- updateDamageCase(id, damageCase)
- getNextCaseNumber(pool, now)
- createUserCode(username)

Material-/Leistungskatalog:

- createCatalogItem(item)
- listCatalogItems(onlyActive)
- updateCatalogItem(id, item)
- deleteCatalogItem(id)

Katalogpositionen am Fall:

- addCatalogItemToCase(caseId, catalogId, menge)
- getCatalogItemsForCase(caseId)
- removeCatalogItemFromCase(posId)

Versicherungskatalog:

- createInsuranceCatalogEntry(entry)
- listInsuranceCatalogEntries(onlyActive)
- updateInsuranceCatalogEntry(id, entry)
- deleteInsuranceCatalogEntry(id)

Kassenkatalog:

- createCashDeskEntry(entry)

- `listCashDeskEntries (onlyActive)`
- `updateCashDeskEntry (id, entry)`
- `deleteCashDeskEntry (id)`
- `hasActiveCashDesk (name)`

Konfiguration:

- `getDbConfig (scope)`
- `resetPoolsForScope (scope)`

6.5 Beispiel Fallnummernlogik

```
async function getNextCaseNumber(pool, now) {
  const yearToken = String(now.getFullYear()).slice(-2);
  const likePattern = `01/${yearToken}%`;

  const result = await pool.query(
    'SELECT MAX("SFANUMMER") AS "maxNumber" FROM "DATSCHADENSFAELLE" WHERE "SFANUMMER" LIKE $1',
    [likePattern],
  );

  const currentSequence = result.rows[0]?.maxNumber
    ? parseInt(String(result.rows[0].maxNumber).slice(-4), 10)
    : 0;

  return `01/${yearToken}${String(currentSequence + 1).padStart(4, "0")}`;
}
```

7. Schadensfall-Workflow

Status im Backend:

- Neu
- Team
- Leitung
- Abgeschlossen

7.1 Anlegen

Beim Anlegen wird auf Basis `costsComplete` entschieden:

```
if (payload.costsComplete) {
  payload.status = "Team";
} else {
  payload.status = "Neu";
}
```

7.2 Voraussetzungen für "Kosten komplett"

Servervalidierung:

```
function isValidCostsCompletePrerequisites(damageCase) {
  const hasResponsibleParty = Boolean(String(damageCase.responsibleParty || "").trim());
  const isValidResponsiblePartyAddress = Boolean(String(damageCase.responsiblePartyAddress || "").trim());
  const hasInsurance = Boolean(String(damageCase.insurance || "").trim());
  const hasCashDesk = Boolean(String(damageCase.cashDesk || "").trim());
  const hasAccidentLocation = Boolean(String(damageCase.street || "").trim());
  const hasPlateNumber = Boolean(String(damageCase.plateNumber || "").trim());

  return (
    hasResponsibleParty &&
    isValidResponsiblePartyAddress &&
    hasInsurance &&
    hasCashDesk &&
    hasAccidentLocation &&
    hasPlateNumber
  );
}
```

7.3 Weiterleitung/Freigabe

- forwardToLeitung + **Recht** forward_to_leitung -> **Status** Leitung
- approve + **Recht** approve_case -> **Status** Abgeschlossen

7.4 Gesperrte Kostenfelder nach Abschluss

Nach costsComplete = true sind relevante Kostenfelder nicht mehr änderbar.

8. Lookup- und Hilfsfunktionen

8.1 Fahrzeug-Lookup

- GET /api/lookup/vehicle?plate=...
- nutzt interne VEHICLE_DATA

8.2 Strassen- und Adress-Lookup

- GET /api/lookup/street
- GET /api/lookup/address

Internet-Lookup über Nominatim mit Timeout/Fallback:

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);
```

Bei transienten Fehlern wird auf lokale Fallback-Daten gearbeitet (wo verfügbar).

9. API-Referenz nach Bereich

9.1 UI-Seiten

- GET /login

- GET /overview
- GET /create-damage-case
- GET /rights
- GET /catalog
- GET /insurance-catalog
- GET /cash-desk-catalog
- GET /db-config

9.2 Schadensfälle

- GET /api/damage-cases?scope=own|department|all
- GET /api/damage-cases/:id
- POST /api/damage-cases
- PUT /api/damage-cases/:id

9.3 Katalogpositionen am Fall

- GET /api/damage-cases/:id/catalog-items
- POST /api/damage-cases/:id/catalog-items
- DELETE /api/damage-cases/:id/catalog-items/:posId

9.4 Material-/Leistungskatalog

- GET /api/catalog (aktiv)
- GET /api/admin/catalog
- POST /api/admin/catalog
- PUT /api/admin/catalog/:id
- DELETE /api/admin/catalog/:id

9.5 Versicherungskatalog

- GET /api/insurance-catalog (aktiv)
- GET /api/admin/insurance-catalog
- POST /api/admin/insurance-catalog
- PUT /api/admin/insurance-catalog/:id
- DELETE /api/admin/insurance-catalog/:id

9.6 Kassenkatalog

- GET /api/cash-desks (aktiv)
- GET /api/admin/cash-desks
- POST /api/admin/cash-desks
- PUT /api/admin/cash-desks/:id
- DELETE /api/admin/cash-desks/:id

9.7 Rechte-/Benutzerverwaltung

- GET /api/users
- GET /api/rights/group-mappings
- PUT /api/rights/group-mappings

9.8 DB-Konfiguration

- GET /api/db-config
- PUT /api/db-config
- POST /api/db-config/test

10. Frontend-Bestandteile

10.1 Seiten

- public/login.html: AD + Dummy Login
- public/overview.html: Fallliste, Scopes, Admin-Links
- public/create_damage_case.html: Detailmaske inkl. Katalogpositionen
- public/rights.html: Rechteverwaltung
- public/catalog.html: Material/Leistung
- public/insurance-catalog.html: Versicherungen
- public/cash-desk-catalog.html: Kassen
- public/db-config.html: DB-Einstellungen

10.2 Zentrale Client-Logik (src/create-damage-case.js)

State-Objekt enthält u. a.:

- cases
- catalogItems
- insuranceCatalog
- cashDeskCatalog
- responsiblePartyAddressValid

Beispiel Kostenvoraussetzungen im Frontend:

```
function isValidCostsCompletePrerequisites() {
  const hasResponsibleParty = Boolean(elements.responsibleParty.value.trim());
  const hasInsurance = Boolean(elements.insurance.value.trim());
  const hasCashDesk = Boolean(elements.cashDesk.value.trim());
  const hasAccidentLocation = Boolean(elements.street.value.trim());
  const hasPlateNumber = Boolean(elements.plateNumber.value.trim());

  return (
    hasResponsibleParty &&
    hasInsurance &&
    hasCashDesk &&
    hasAccidentLocation &&
    hasPlateNumber
  );
}
```

11. Guarding und Sicherheit

Wichtige Guards im Server:

```
function requireLogin(req, res, next) {
  if (req.session && req.session.user) return next();
  if (req.path.startsWith("/api/")) return res.status(401).json({ error: "Nicht angemeldet" });
  res.redirect("/login");
}

function requirePermission(permission) {
  return (req, res, next) => {
    if (rights.hasPermission(req.session?.user, permission)) return next();
    res.status(403).json({ error: "Zugriff verweigert", missingPermission: permission });
  };
}
```

Technische Hinweise:

- Session-Cookie aktuell mit `secure: false` (lokal ohne HTTPS möglich)
- produktiv sollte HTTPS mit gültigen Zertifikaten aktiv sein
- Fehler werden serverseitig geloggt, API liefert gezielte Fehlermeldungen

12. Wichtige Entwicklungsregeln

1. Rechte nur in `src/rights.js` erweitern, nicht in Endpunkten duplizieren.
2. SQL-Zugriffe nur in `src/db.js` pflegen.
3. Neue Workflow-Status immer in beiden Schichten anpassen:
 - Rechteprüfung (`canViewCase`, `canEditCase`)
 - API-Workflow (`POST/PUT /api/damage-cases`)
4. Neue Felder für Schadensfälle in drei Stellen synchron halten:
 - `sanitizeDamageCaseInput` in `server.mjs`
 - Persistenz in `src/db.js`
 - Form-Bindings in `src/create-damage-case.js`
5. Kataloge bevorzugt soft-löschen, damit Historie konsistent bleibt.

13. Typischer End-to-End-Ablauf

1. Login über `/login` (Dummy oder AD).
2. Rechteableitung über LDAP-Gruppen und Rollenmapping.
3. Session-User wird gesetzt, Benutzer via `db.upsertUser` synchronisiert.
4. Übersicht lädt Fälle via `/api/damage-cases`.
5. Detailseite lädt Fall und Katalogpositionen.
6. Bei Änderungen greifen Status-/Rechte-/Validierungsregeln serverseitig.
7. Katalog-, Versicherungs- und Kassenstammdaten werden über Admin-APIs gepflegt.

14. Kurzfazit

SaS3 ist als klare 3-Schichten-Struktur umgesetzt:

- API/Workflow in `server.mjs`
- Rechte in `src/rights.js`
- Datenhaltung in `src/db.js`

Die wichtigsten Erweiterungspunkte sind bereits zentralisiert. Dadurch lassen sich neue Fachregeln, Felder und Rollen ohne verteilte Sonderlogik implementieren.

15. Offene Punkte (bewusst nicht umgesetzt)

Die folgenden Themen sind im aktuellen Stand als nächste Ausbaustufe vorgesehen:

- Dateiupload inkl. revisionssicherer Ablage
- SAP-Schnittstelle für Rechnungsübergabe und Rückmeldung zum Zahlungseingang

Hinweis für die technische Umsetzung:

- Beide Themen betreffen API, Berechtigungslogik, Persistenz und Betriebskonzept.
- Bei Umsetzung sollten Integrations- und Regressionstests für den kompletten Workflow ergänzt werden.